

```
Testing all functions for n=2...
n=2: 14/16 separable (0.8750)

Testing all functions for n=3...
n=3: 104/256 separable (0.4062)

Sampling 10000 functions for n=4...
n=4: 285/10000 separable (estimated: 0.0285)

Sampling 10000 functions for n=5...
n=5: 0/10000 separable (estimated: 0.0000)

Results show that the fraction of linearly separable
Boolean functions decreases rapidly with n:
n=2: 87.50%
n=3: 40.62%
n=4: 2.85%
n=5: 0.00%
```

Figure 1: Tests for $n=2,3,4,5$ with 20 epochs and learning rate = 0.05

Method/Algorithm Description: I implemented a single-layer perceptron to test the linear separability of Boolean functions across different input dimensions ($n=2,3,4,5$). The algorithm uses gradient descent with 20 epochs and a learning rate of 0.05. For smaller dimensions ($n=2,3$), all possible Boolean functions were tested, while for larger dimensions ($n=4,5$), I sampled 10000 random functions due to computational limitations. To create input patterns I take the number of the current iteration and do a floor division, that it is equivalent to a bit shift to the right, so an 16 becomes a 8, 8 ends to a 4 and so on. I use it to create the pattern with 0 and 1. Same thing with the boolean functions, but to convert 0 and 1 into -1 and 1 I use the modulus operator to know if the value is odd or even.

Results Summary: From the result we can see the exponential decay in linear separability as input dimension increases. With multiple instances of the algorithm I noticed that for $n = 2$ and 3 the output was always the same. I had differences for $n = 4$ and $n = 5$, in particular the maximum in $n = 5$ was $1/10000$ instead of 0.

Error Analysis and Inaccuracies: This variation in the code is caused by sampling: in my code I sample 10000 random functions and not all of them. Random initialization: the value of the weights at the start are randomly chosen, a solution in this case is by increasing the number of samples but with a cost in term of computation. For a small amount a cause of variation could be also the floating-point arithmetic that is not exact in Python.

Conclusions: Single-layer networks are good when we don't have a big amount of boolean functions, so small input functions but for $n=4$ and so on it's better to use a multi-layered network.